Homework 2 — LLM Chat

Technical Report

Author: Lei Yaqi

Stack: Python, FastAPI, MongoDB, Redis, Jinja2, llama-cpp-python / Ollama / OpenAI

1. Frontend & Architecture

Frontend: server-rendered HTML (Jinja2 templates).
Backend: Model-View-Controller (MVC).

Layer	Implementation
Model	MongoDB collections via PyMongo
View	templates/index.html, templates/chat.html + static/app.css
Controller	FastAPI route files (api/auth.py, api/chats.py, web/pages.py)

2. API Endpoints

All API routes under /api, JSON, protected routes require Authorization: Bearer <access_token>.

2.1 Authentication (`/api/auth`)

Method	Path	Request	Response
POST	/register	{ login, password }	{ id, login }
POST	/login	{ login, password }	{ access_token, refresh_token, token_type }
POST	/refresh	{ refresh_token }	{ access_token, token_type }

2.2 GitHub OAuth

Method	Path	Behavior
GET	/auth/github/login	returns { authorize_url }
GET	/auth/github/callback	exchanges code, redirects to /chat?access_token=...&refresh_token=...

2.3 Chats (`/api/chats`)

Method	Path	Response
GET	/api/chats	ChatOut[]
POST	/api/chats	ChatOut
GET	/api/chats/{chat_id}	ChatWithMessagesOut
POST	/api/chats/{chat_id}/messages	LlmAnswerOut (user + assistant messages)
DELETE	/api/chats/{chat_id}	{ ok: true }

2.4 LLM status

Method	Path	Response
GET	/api/llm/status	{ configured, openai, ollama, llama_cpp, ... }

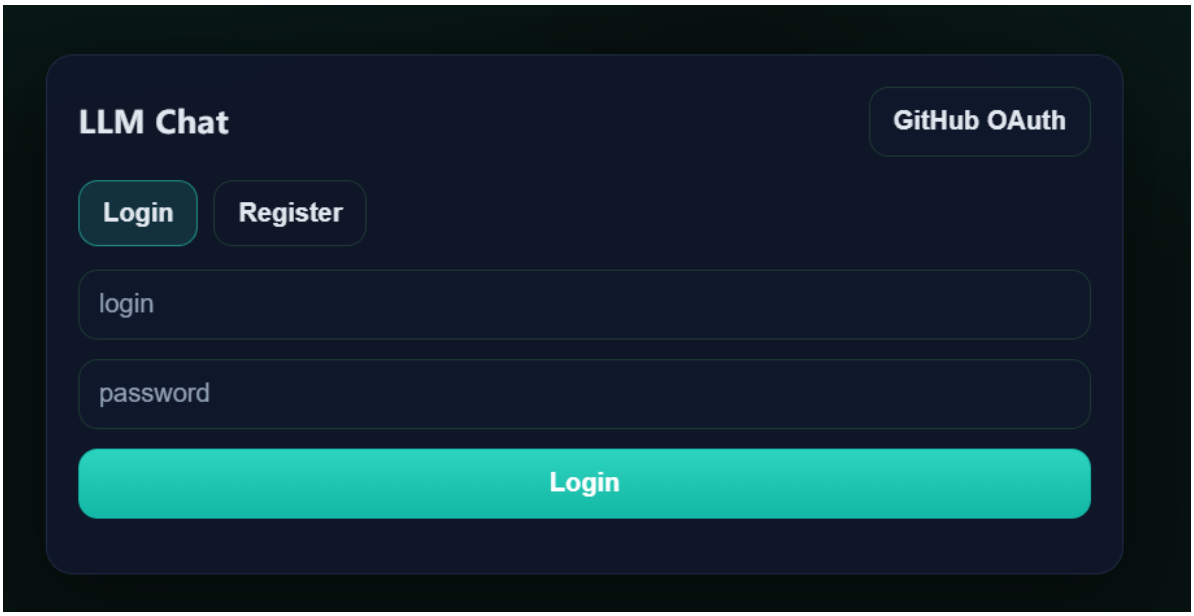
2.5 Page routes

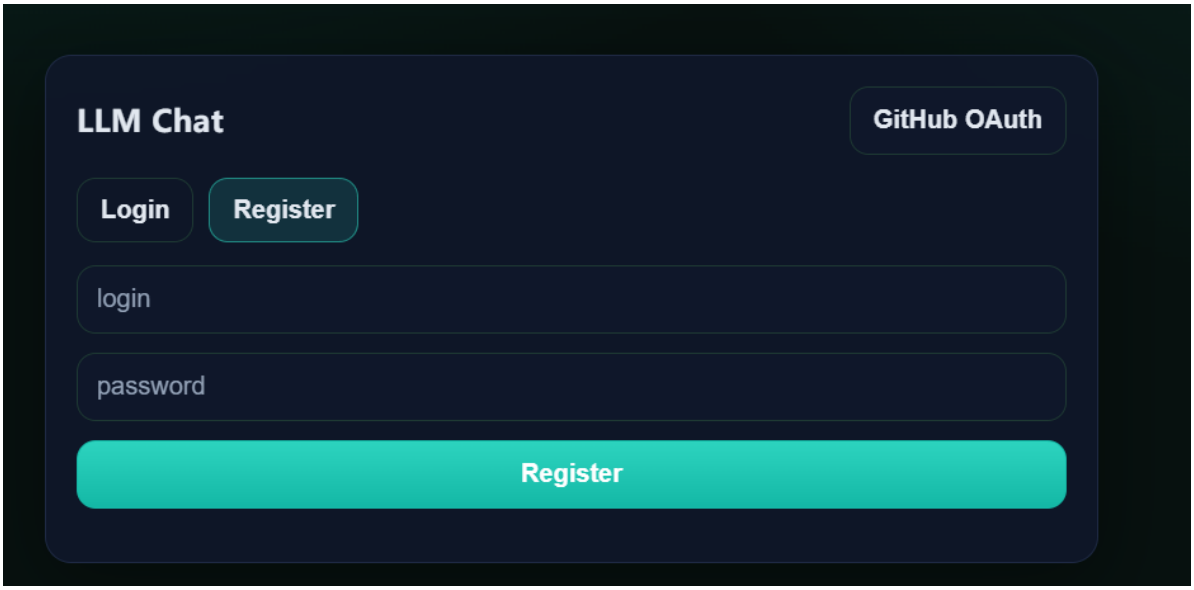
Method	Path	Template
GET	/	index.html (login/register)
GET	/chat	chat.html (main chat UI)

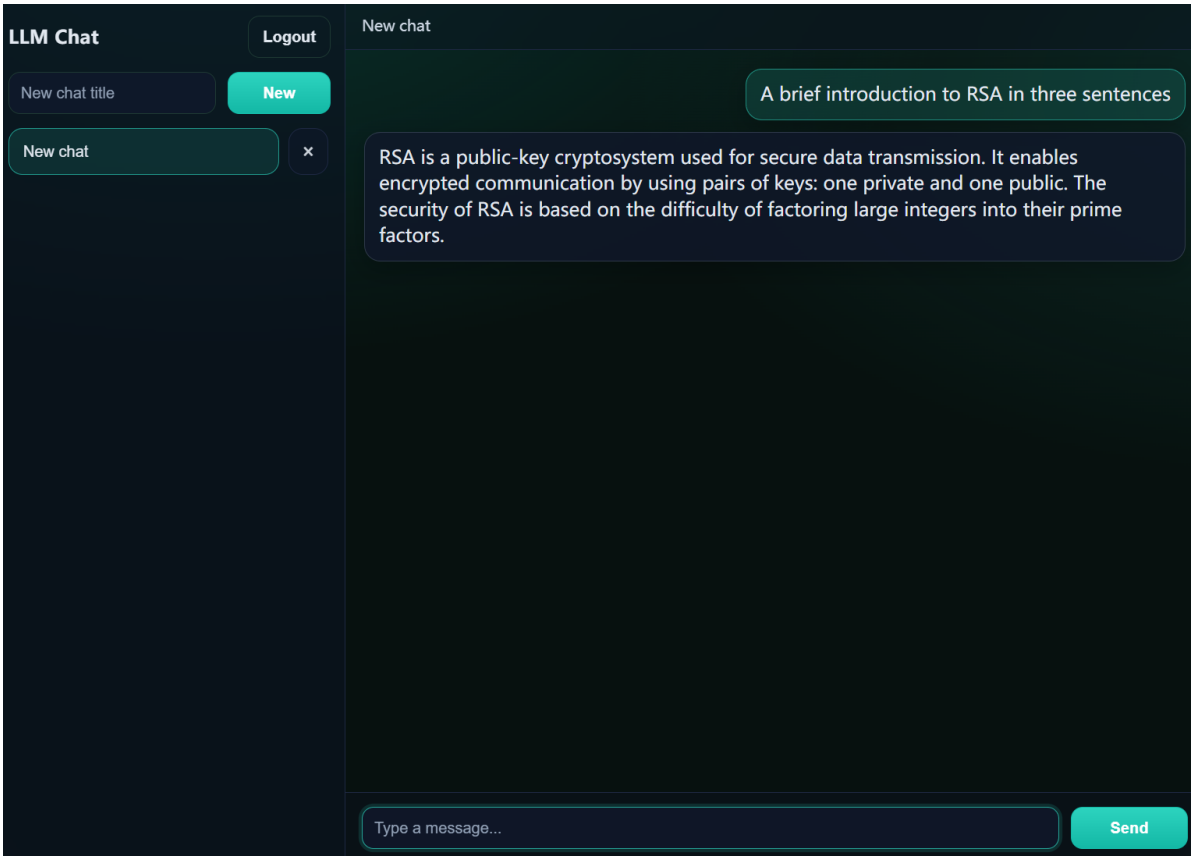
3. Codebase Structure

```
.
├── main.py
├── db.py                # MongoDB connection + index creation
├── redis_client.py      # Redis connection (supports fakeredis)
├── settings.py          # env vars (dataclass)
├── security.py          # password hashing, JWT encode/decode
├── deps.py              # get_current_user (JWT validation)
├── schemas.py           # Pydantic models
├── api/                 # Controllers
|   ├── auth.py
|   ├── chats.py
|   └── llm.py
├── services/            # Business logic + data access
|   ├── users.py
|   ├── chats.py
|   ├── auth.py
|   ├── oauth_github.py
|   └── llm.py
├── web/                 # View layer
|   └── pages.py
├── templates/
|   ├── index.html
|   └── chat.html
└── static/
    └── app.css
```

4. UI Screenshots







Click GitHub OAuth

Sign in to **GitHub**
to continue to **LLM-chat**

Username or email address

Password Forgot password?

Sign in

or

Continue with Google

Continue with Apple

New to GitHub? Create an account

Sign in with a passkey

4.1 / – Login / Register

- Two tabs: Login (login+password) / Register (login+password)
- GitHub OAuth button
- After login: tokens stored in localStorage, redirect to /chat

4.2 /chat – Main chat interface

- Sidebar: brand, Logout, "New chat" input+button, list of chats (each with delete button)
- Main area: current chat title, message history (user messages right-aligned, assistant left), input + Send button
- Sending: shows user message, loading spinner, calls /api/chats/{id}/messages, displays assistant reply
- 401 handling: auto-refresh using refresh token

5. Database Schema (MongoDB)

users

- _id: ObjectId
- login: string, unique index
- password_hash: string (bcrypt/pbkdf2) – empty for GitHub users
- github_id: integer, sparse unique index

chats

- _id: ObjectId
- user_id: ObjectId (ref users._id)
- title: string
- created_at: ISO string
- Index: (user_id, created_at DESC)

messages

- _id: ObjectId
- chat_id: ObjectId (ref chats._id)
- user_id: ObjectId (denormalized)
- role: 'user' or 'assistant'
- content: string
- created_at: ISO string
- Index: (chat_id, created_at ASC)

6. JWT + Refresh Token + Redis

- Access token: JWT (HS256), payload contains sub (user_id) and login, TTL = ACCESS_TTL_SECONDS (default 900s)
- Refresh token: random 32-byte URL-safe string, stored in Redis as key refresh: -> user_id, TTL = 30 days (REFRESH_TTL_SECONDS)
- Refresh flow: client calls /api/auth/refresh with refresh token, receives new access token
- OAuth state: stored in Redis as oauth_state: -> "1", TTL 10 minutes, consumed after callback
- Logout: client clears localStorage; no server-side revocation (refresh token expires naturally)

7. LLM Integration

`services/llm.py` provides `answer_chat(messages)` with fallback: OpenAI → Ollama → llama-cpp-python.

Backend	Environment variables	Notes
OpenAI	OPENAI_API_KEY, OPENAI_BASE_URL, OPENAI_MODEL	Uses /v1/chat/completions
Ollama	OLLAMA_URL, OLLAMA_MODEL	Auto-detects available models, falls back to first found
llama-cpp	MODEL_PATH	Loads .gguf, n_ctx=512, n_threads=4, non-streaming

- Last 12 messages of current chat sent as context (converted to {role, content} list)
- For llama-cpp (no native chat format): concatenates as "User: ...\nAssistant: ..."
- GET /api/llm/status checks configuration and reachability

---

8. Running the Project

Environment variables

Variable	Purpose
JWT_SECRET	Required, any string
MONGODB_URI	mongomock://localhost (mock) or mongodb://localhost:27017 (real)
REDIS_URL	fakeredis:// (mock) or redis://localhost:6379/0 (real)
BASE_URL	e.g. http://127.0.0.1:8000
OLLAMA_URL	http://127.0.0.1:11434 (optional)
OLLAMA_MODEL	e.g. qwen2.5:7b (optional)

Mode A – No external dependencies (mock)
Uses mongomock and fakeredis.

bash

```
export JWT_SECRET="dev-secret"
export MONGODB_URI="mongomock://localhost"
export REDIS_URL="fakeredis://"
export BASE_URL="http://127.0.0.1:8001"
export OLLAMA_URL="http://127.0.0.1:11434"
export OLLAMA_MODEL="qwen2.5:7b"

uvicorn app.main:app --host 127.0.0.1 --port 8001
```

Mode B – Real MongoDB + Redis

bash

```
export JWT_SECRET="dev-secret"
export MONGODB_URI="mongodb://localhost:27017"
export MONGODB_DB="wad_homework_2"
export REDIS_URL="redis://localhost:6379/0"
export BASE_URL="http://127.0.0.1:8001"
export OLLAMA_URL="http://127.0.0.1:11434"
export OLLAMA_MODEL="qwen2.5:7b"

uvicorn app.main:app --host 127.0.0.1 --port 8001
```

Ollama (local LLM)

bash

```
ollama serve
ollama run qwen2.5:7b
```

Check LLM status: `GET http://127.0.0.1:8001/api/llm/status` returns `"ollama": true` when connected.